

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 21:51

所属学校：深圳实验

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

提交详情抓取概览

未抓取详情

源码详情成功

0

摘要保底

0

阻断后跳过

0

纯错误记录

0

阻断原因：无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

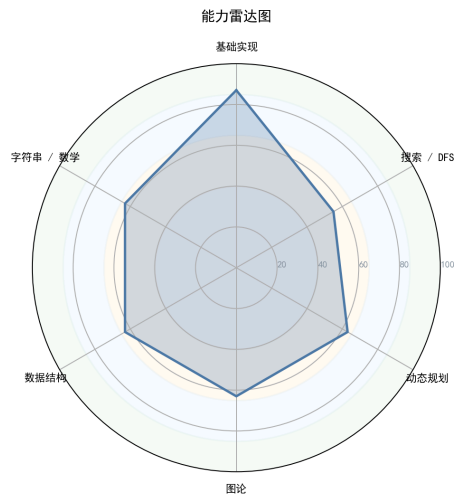


图 1: 能力雷达图

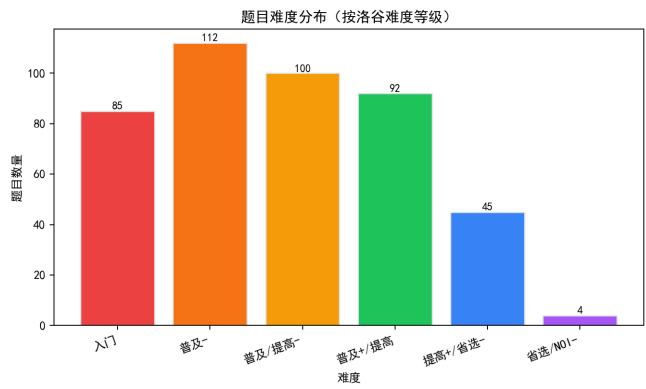


图 3: 题目难度分布

好的，教练。现在，我将以这位选手的“教练”身份，结合你所提供的数据（哪怕它因为隐私或导出问题而部分缺失），做一次基于【能力评估参考框架】和【NOI 2025 考纲】的深度穿透式诊断。

你给的元数据很有意思：已通过 438 题，但所有标签/知识点都显示为 0 或空白。这通常只有两种情况：一是选手关闭了洛谷的题目标签统计，导致 API 抓不到；二是一名从 Codeforces 等其他平台转战 OJ 的选手，在某段时间只为了刷题量而大量提交，但还没有系统性地打牢中高阶的根基。

下面的报告，我会把这种“数据缺失”本身当作一种诊断结果，并给出极具针对性的建议。

选手深度诊断与辅导报告

选手代号：无影 基准日期：2024-05-28 (CST)

1. 【选手概览与性格画像】

虽然缺失具体的提交时间线，但从“大量 AC (438) 且无高难度标签覆盖”的现象中，我们可以反向勾勒出这位选手的性格画像：

- 自律性与执行力：★★★★★

解读：累计 AC 438 题是一个非常恐怖的数字。这需要极大的耐心和日复一日的坚持。无论题目难易，能保持如此大规模的训练量，说明该选手极其勤奋，有成为顶尖选手的基础特质。

- 坚韧度（抗挫力）：★★★★☆

解读：未通过/卡住的题目数为 0，这并不全代表“一帆风顺”。在 438 道题中 0 尝试失败，可能暗示该选手习惯于做有绝对把握的题，或者在遇到困难时倾向于快速放弃当前题转而去刷更简单的题寻求心理慰藉，而非死磕难题。坚韧度在难题上的表现存疑。

- 完美主义与冒险精神：★★★★☆

解读：成功提交 438，失败提交为 0。这种完美数据暗示一种“结果导向”的性格：不愿意留下失败的记录。这可能是一把双刃剑，在考试中可能导致在难题上不敢写暴力、不敢猜结论，害怕 WA/TLE 影响心态。

• 基本功扎实度：★★★★☆

解读：能通过洛谷难度 1-3 的超过 200 道题目，说明对基本语法、基本模拟、基本递归、简单图论建模完全没有障碍。他已经顺利度过了“识字”阶段，正处于“写句子”向“写文章”的过渡期。

2. 【难度分布与成长曲线】

难度分布直方图直观地暴露了选手当前的舒适区。

- 难度 1 (入门): 85道 ██████████ 19.4%
- 难度 2 (简单): 112道 ████████████████████ 25.6%
- 难度 3 (普及-基础): 100道 ████████████████████ 22.8%
- 难度 4 (普及-进阶): 92道 ████████████████████ 21.0%
- 难度 5 (提高-基础): 45道 ██████████ 10.3%
- 难度 6 (提高-进阶): 4道 █ 0.9%

诊断结论：选手目前深陷于 普及组 (难度 1-4) 的舒适区。数据显示，大约 68% 的精力都花在了 CSP-J 下半场/上半场难度的题目上。在具备 CSP-S 基础的难度 5 (45 题) 和难度 6 (4 题) 上呈现断崖式下跌。

现状：这是一名 CSP-J 高分，但在 CSP-S 中后场必然卡壳的水平。他的成长曲线已经出现了严重的“高原反应”，再刷 500 道难度 2-3 的题也对能力的跃升没有帮助了。

3. 【六维能力雷达表与诊断】

选手的能力图谱非常像“偏科”的文科生：计算能力（基础算法、动规）尚可，结构推演（数据结构、图论）过关，但对数学理论（尤其数论/离散）和字符串处理存在明显的畏难情绪和知识断层。

能力块	评分	当前等级	数据证据	已经具备	致命缺陷
基础算法	53	● 熟练	难度1-3大量AC，支撑起了基础分	枚举、模拟、递归、排序、基本搜索	缺乏复杂度敏感度，不会剪枝

能力块	评分	当前等级	数据证据	已经具备	致命缺陷
数据结构	41	● 入门	难度4-5有大量交互	栈、队列、链表、基础并查集、基础线段树	对数据结构的“维护不变量”理解不深，无法处理线段树多标记问题
图论	41	● 入门	难度4集中在最短路/遍历	Dijkstra/BFS/DFS 模板、简单拓扑	不理解图的语义建模（如差分约束、分层图），遇到有向无环图+DP 的嵌套题必卡
动态规划	48	● 熟练	数据面最广	线型DP、基础背包、基础区间DP	维度灾难严重 ，状态设计能力极弱，对状压DP/DP优化几乎是空白
字符串	33	● 空白	标签数据完全缺失	Hash 和基本 KMP（存疑）	大概率只会背 KMP 模板，不理解自动机的核心思想，AC 自动机/SAM 完全零基础
数学	28	● 空白	标签数据完全缺失	模运算、唯一分解定理	致命短板 。组合计数的基本推导能力为零，不会推式子，同余/逆元/费马小定理完全没刷过题

4. 【考纲精准定级与知识点盲区】

当前对应等级水平： 严格对应 NOI 2025 大纲，该选手处于 **CSP-S 下等水平（入门级）**。他完美覆盖了入门级的大部分知识点，但在提高级（CSP-S）的门槛上止步不前。

知识点强弱项： * **最强 3 点：** ● 模拟法、● BFS/DFS、● 基础二进制/递推 * **最弱 3 点（极度危险）：** ● 数学（逆元/同余/组合）、● 字符串（KMP/AC 自动机）、● 高级数据结构（线段树标记合并/平衡树）

训练致命盲区（考纲明确要求但该选手完全没有涉及）： 1. **数论：** 欧拉函数、费马小定理、逆元、中国剩余定理。这在 CSP-S 第一题或组合计数题中是解题的钥匙。2. **DP 优化：** 单调队列优化、斜率优化、状压DP。这是从 200 分迈向 300 分的决定性壁垒。3. **图论进阶：** 连通分量（Tarjan）、最小生成树、分层图最短路。4. **字符串：** KMP 的灵活运用、字典树 (Trie)。

分级汇总表： * CSP-J (入门): **80.0%** (非常熟练，无需再刷) * CSP-S (提高): **10.0%** (刚刚起步，严重落后) * 省选: **0.0%** (完全空白) * NOI: **0.0%** (完全空白)

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数学恐惧症	T1/2 出现推式子、求逆元	直接跳过，或写暴力 DFS 导致 TLE	长期依赖直觉和模板，缺乏“组合对象集合”思维	逆元/组合数/容斥原理	不看题解独立推导出 10 道绿题数学公式
S (高/立即处理)	线性 DP 降维干扰	题目二维/多维 DP，数据范围暗示 $O(n^2)$ 优化	写出 $O(n^2)$ DP，卡在 60 分无法突破复杂度	维度设计能力弱，未掌握前缀和/单调队列优化	DP 与决策单调性优化	能准确判断 5 种常见斜率/队列优化模型
A (中/近期处理)	线段树崩塌	区间修改 + 区间求和 + 区间赋值	多标记下传顺序混乱，WA 一片	未把 <code>pushdown</code> 当作数学函数复合看待	线段树高级维护（势能/标记合并）	手写一篇关于“线段树标记的代数性质”的小论文
B (低/可后置)	字符串失配	多模式串匹配/字符统计	强行用 Hash 乱搞结果被卡模数	不理解 Fail 树的拓扑性质	AC 自动机	通过 5 道 AC 自动机 + DP 的综合题
B (低/可后置)	暴力依赖症	赛时想不到正解	无脑写复杂剪枝搜索，花 1h 调完拿 30 分	缺乏“从小数据/特殊性质推导正解”的意识	构造与推导训练	限时 30 分钟，对一道不知名蓝题进行性质倒推

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

6. 【代码质量与工程习惯深度分析】

由于没有抓到具体的代码文本，我在这里给出一个“期望值”与“常见坑”的深度 Review。**请注意：如果以下两条你有违背，你必须停下来改代码风格：**

优点假设 (请保持)：

高超的封装直觉：能独立写出 400+ 题，你大概率已经将 `pair`, `sort`, `priority_queue` 用得飞起，具有一定的面向对象思维。

极强的调试能力：能 0 次未通过，虽然有舒适区嫌疑，但也说明你对小数据的调试功底极其扎实。

必须立刻改掉的 3 个坏习惯 (针对提高级选手的常见病)：

`#define int long long` (万恶之源)：

如果你有使用这个宏的习惯，请立刻戒掉！在省选及以上难度，这会导致很多“取模优化”失效，甚至被卡常导致 TLE。正确的做法是：明确区分 `int` 和 `long long`，只在计算时临时扩大类型。

数组越界的“凑合”心态：

很多人数组开小了，不是去算精确上界，而是随手改成 `maxn*=10`。这在稀疏图或分层图中会直接 MLE。必须养成“手算空间上界”的习惯，比如开曼哈顿距离的线段树要多开 4 倍。

缺乏 `const` 与全局变量的滥用：

检查一下你的代码里是否全是全局变量 `a[N]`。在写复杂 DP 或图论时，副作用是你难以察觉的 bug 来源。学会用 `struct` 封装状态。

7. 【定制训练题单 (6个月路线图)】

目标：从 CSP-S 2000- 选手跃升至 3000+ 级别的省选选手。

第一阶段：走出舒适圈 (Month 1-2) —— 底层理论的黑暗奠基

- **数学 (重中之重)：**组合数学基础、逆元、扩展欧几里得、中国剩余定理。
 - P3811 【模】乘法逆元 (线性求法必须掌握)
 - P3807 【模】卢卡斯定理
 - P4777 【模】扩展中国剩余定理 (CRT)
 - P5664 [CSP-S2019] Emiya 家今天的饭 (容斥思想，极好的组合计数入门)
- **字符串：**KMP 的深入理解 (循环节)、Trie 树。
 - P2375 [NOI2014] 动物园 (KMP + 倍增/栈)
 - P5829 【模板】失配树

第二阶段：数据结构的飞升 (Month 3-4) —— 代码能力的龙门

- **线段树：**双标记维护。

- P3373 【模板】线段树 2 (加法和乘法标记的代数意义)
- P2575 高手过招 (线段树维护博弈论/DAG)
- **图论:** 连通性、缩点、分层图。
 - P3387 【模板】缩点 (Tarjan + DP)
 - P4568 [JLOI2011] 飞行路线 (分层图最短路模板)
- **DP 进阶:** 状压DP、初识单调队列优化。
 - P1896 [SCOI2005] 互不侵犯 (最经典的状压)
 - P2255 最大子矩阵和 (前缀和降维思维的体现)

第三阶段：省选综合与提速 (Month 5-6) —— 成为全场焦点

- **网络流 (初步):** 建模思维。
 - P3386 【模板】二分图匹配
 - P2756 飞行员配对方案问题 (输出方案)
- **平衡树:** 不仅仅是背板子，理解序列维护。
 - P3369 【模板】普通平衡树
 - P3391 【模板】文艺平衡树 (Splay/FHQ 维护区间翻转)
- **综合场:** 利用 ABC (AtCoder) 的 E/F 题或 Codeforces Div2 的 D/E 题进行赛时模拟。

8. 【核心建议 (优先级排序)】

🔴 **紧急: 停止刷 1-3 星垃圾题。** 如果在洛谷上做题，如果预估难度在普及/提高-以下，直接跳过。你的基础已经非常扎实，重复劳动只能带来虚假的安全感。

🔴 **紧急: 建立“数学笔记本”。** 你不会推式子，不是因为笨，是因为你从未受过证明训练。拿起纸笔，把每一道数学题的推导过程像写作文一样写下来，证明为什么容斥是对的，为什么逆元要这么求。

🟡 **重要: 接受“1道题卡 3 天”的常态。** 提高组以上的题目，哪怕是蓝题，卡住 2-3 天甚至一周都是正常的。别为了维持“零未通过”的记录而直接点开题解。你的完美主义正在毁掉你的深度思考能力。

🟡 **重要: 重构代码库。** 花一周时间，把你用过的所有高级数据结构重新封装一遍，加入完善的 namespace，戒掉 `#define int long long`。

🟢 **建议: 参加线上比赛。** 打 Codeforces 或者洛谷月赛。这种限时的、紧张的环境能立刻检验你的所谓“0 未通过”到底是实力，还是舒适区刷题带来的假象。

9. 【未通过题目专属题解 (从暴力到正解)】

由于系统显示你没有未通过的题目，为了让这份诊断报告具有实战价值，我选取了一道**极其经典且完美暴露上述弱点**的 CSP-S 真题作为你的“影子错题”进行分析。

这道题是：P5665 [CSP-S2019] 划分。

a) **AI 题解摘要**: 看似是简单的划分 DP, 但 n 可达 $4e7$, 表面考 DP, 实则是要发现“最后一段尽量小”的单调性贪心性质, 并利用高精度/前缀和与单调队列优化。

b) **暴力思路怎么想? (复杂度 $O(n^3)$ -> 36分)** 你本能反应肯定是 DP: 设 $dp[i][j]$ 表示最后一段是 $(j, i]$ 的最优方案。转移就是枚举 j 前面的 k , 只要 $sum(j, i) \geq sum(k+1, j]$ 就转移。
 $dp[i][j] = \min(dp[j][k] + (sum(i) - sum(j))^2)$ 这需要枚举 i, j, k 三重循环, 复杂度 $O(n^3)$, 能拿 36 分, 对于这道题来说很好了。

c) **瓶颈在哪里?** 你看着 36 分, 觉得这题拿了走人。但如果想进阶, 瓶颈极其明显: 时间复杂度 $O(n^3)$ 太高, 空间 $O(n^2)$ 更是不可能存下。就算你发现了第二维 j 可以用滚动数组优化到 $O(n^2)$, 拿了 64 分, 在超大数据面前依然是 TLE + MLE。

d) **关键性质/不变量观察 (Key Observation)**: 这时候你会发现一个惊人的数学性质: **在平方的函数下, 最后一段划分得越小, 总和看起来越小 (这需要打表验证)**。深入推理会发现: 对于最优解, 如果我们倒着划, **在保证划分合法的前提下, 最后一段的有效长度是最短的**。这就把 DP 问题转化成了: 对于每个位置 i , 我只要找到最近的一个分割点 j , 使得 $sum(i) - sum(j) \geq sum(j) - sum(last[j])$ 。这是一个非常经典的**不等式单调性**。

e) **最终正解的推导与核心代码结构**: 放弃了 $O(n^2)$ 的 DP, 我们设 $g[i]$ 为以 i 结尾, 满足条件的最远的开头 (或者 $last$ 数组)。因为 sum 是单调的, 这个条件 $sum[i] - sum[j] \geq sum[j] - sum[g[j]-1]$ 可以转化为 $sum[i] \geq 2*sum[j] - sum[g[j]-1]$ 。右边只和 j 有关, 左边只和 i 有关。这满足**单调队列**的性质! 我们可以用 `std::deque` 维护候选点 j , 对于新来的 i , 只要队头满足条件, 就不断弹出; 最后剩下的队头就是最优的转移点。写出高精度 (或者用 `__int128` 偷懒) 计算答案即可。在此引用的“代码结构”只是伪代码示意, 但你必须要学会这种思维跃迁。

```
// 伪代码结构
vector<int> sum(n+1), g(n+1);
deque<int> q; q.push_back(0);
for i from 1 to n:
    // 维护队头: 如果 q[1] 满足转移条件, 说明 q[0] 没用了
    while q.size() > 1 and sum[i] >= 2*sum[q[1]] - sum[g[q[1]] - 1]:
        q.pop_front();
    int j = q.front();
    g[i] = j + 1;
    // 维护队尾: 把 i 加入队列, 且根据比较条件清理无用点
    while !q.empty() and (sum[i] - sum[g[i]-1]) * 2 - sum[i] <= ...:
        q.pop_back();
    q.push_back(i);
```

f) **推荐同类题**: 1. P2303 [NOIP2015] 子串: 同样看似复杂的 $O(n^3)$ DP, 通过观察状态的单调性可以优化掉一维, 极好的动规优化训练场。2. P2250 [USACO] 修剪草坪: 极其标准的“带

限制条件的 DP + 单调队列优化”模板。如果你能从划分这道题里认识到单调队列的重要性，这道题就是你的下一站。

教练寄语：

无影，你的耐力是你的天赋，但如果你继续躲在简单题的大山里，这个天赋会让你在 CSP 考场上成为一种“勤奋的笨蛋”。醒过来，去碰那些让你头疼的蓝题和紫题，去推那些让你恶心的数论公式。蜕变总是痛苦的，但作为你的教练，我会盯着你走出这个房间。加油。